

LeetCode 505 - The Maze II

★★★★★ Pattern: Dijkstra on Rolling Grid

1. Problem Statement

This problem utilizes exactly the same maze as Maze I.

- 0 = Empty
- 1 = Wall

Aadya starts at `start` and aims for `destination`. The primary condition holds: **she cannot stop whenever she wants**. She rolls until she hits a wall or boundary and only then stops.

The Twist:

- **Earlier (Maze I):** Can Aadya reach? (Boolean)
- **Now (Maze II):** What is the *minimum distance travelled*?

2. Why BFS Fails Here

The rolling logic is IDENTICAL to Maze I, but the graph algorithm changes. Why?

Scenario

Suppose the graph behaves like this:

- Path 1: $A \rightarrow \text{roll 2 steps} \rightarrow B$
- Path 2: $A \rightarrow \text{roll 10 steps} \rightarrow C$

BFS behavior: BFS thinks $A \rightarrow B$ is equal to $A \rightarrow C$ because both take exactly **1 move** (one decision in the queue).

Reality: The actual costs are **2** and **10** respectively. They are not equal!

Core Rule: Whenever edges have *Different weights* $\rightarrow$ Use **Dijkstra**.

3. Standard Dijkstra Template vs BFS

You already know BFS. Only three things change in Dijkstra:

Concept	Earlier (BFS)	Now (Dijkstra)
Queue	<code>queue<pair<int,int>> q;</code>	<code>priority_queue<...> pq;</code> (<i>Min-Heap</i>)
Tracking	<code>seen[][]</code>	<code>dist[][]</code>
Insertion	Push once to queue.	May push multiple times. Keep the smaller distance.

Why No seen[][]?

This is the biggest conceptual difference.

Suppose Aadya reaches `(4,5)` with distance **20**. Later, another path reaches `(4,5)` with distance **15**.

Should we ignore it because it's already "seen"? **No**. Because $15 < 20$, this is a better path. Hence, no seen array. We keep `dist[r][c]` instead and only update if we find a strictly shorter distance.

4. Breaking Down the Priority Queue & Relaxation

The Priority Queue

C++ Min-Heap Declaration

```
priority_queue<pair<int,pair<int,int>>, vector<pair<int,pair<int,int>>>,  
greater<pair<int,pair<int,int>>>> pq;
```

Each element of the priority queue is `{distance, {row, col}}`.

So when you write `pq.push({0, {start[0], start[1]}});` it means:

- **Distance** = 0
- **Row** = start[0]
- **Col** = start[1]

Why distance = 0? Because Aadya starts at the starting cell. She has travelled 0 cells so far.

Relaxation Step

Exactly standard Dijkstra:

1. Suppose current distance = **10**
2. Rolling right takes **4** steps.
3. New distance = **14**
4. If **14 < dist[new]**, then update the distance array and push the new state to the priority queue.

5. C++ Implementation

Solution.cpp

```
class Solution {  
public:  
  
    vector<pair<int,int>> dir={{-1,0},{0,-1},{1,0},{0,1}};  
  
    int shortestDistance(vector<vector<int>>& maze, vector<int>& start,  
vector<int>& destination) {
```

```

int n=maze.size();
int m=maze[0].size();

vector<vector<int>> dist(n,vector<int>(m,INT_MAX));
//har ek ka initial distance inf le rhe

priority_queue<
pair<int,pair<int,int>>,
vector<pair<int,pair<int,int>>>,
greater<pair<int,pair<int,int>>>> pq;

pq.push({0,{start[0],start[1]}});
dist[start[0]][start[1]]=0;

while(!pq.empty()){

    int currdist=pq.top().first;
    int r=pq.top().second.first;
    int c=pq.top().second.second;
    pq.pop();

    // Ignore outdated entries.
    if(currdist>dist[r][c]){
        continue;
    }

    for(auto it:dir){

        int nr=r;
        int nc=c;

        int steps=0;

        // EXACT SAME rolling loop as Maze I.
        while(nr+it.first>=0 && nr+it.first<n &&
            nc+it.second>=0 && nc+it.second<m &&
            maze[nr+it.first][nc+it.second]==0){

            nr+=it.first;
            nc+=it.second;
            steps++;
        }

        // Relaxation
        if(currdist+steps<dist[nr][nc]){

            dist[nr][nc]=currdist+steps;

```

```
        pq.push({dist[nr][nc],{nr,nc}});

    }

}

if(dist[destination[0]][destination[1]]==INT_MAX){
    return -1;
}

return dist[destination[0]][destination[1]];
}
};
```